



SECURE KUBERNETES ON AZURE

A private AKS platform — designed, deployed, broken, diagnosed,
and fixed end-to-end on real Azure infrastructure

DEV + PROD · BOTH VERIFIED LIVE

What This Is

A containerized web application running on a private Azure Kubernetes Service cluster, fronted by a WAF-enabled Application Gateway, backed by a private container registry, deployed through OIDC-authenticated GitHub Actions CI/CD — with zero long-lived credentials anywhere in the pipeline.



2

Independent
environments



0

Stored credentials
in the pipeline



PRIVATE

API server —
no public IP

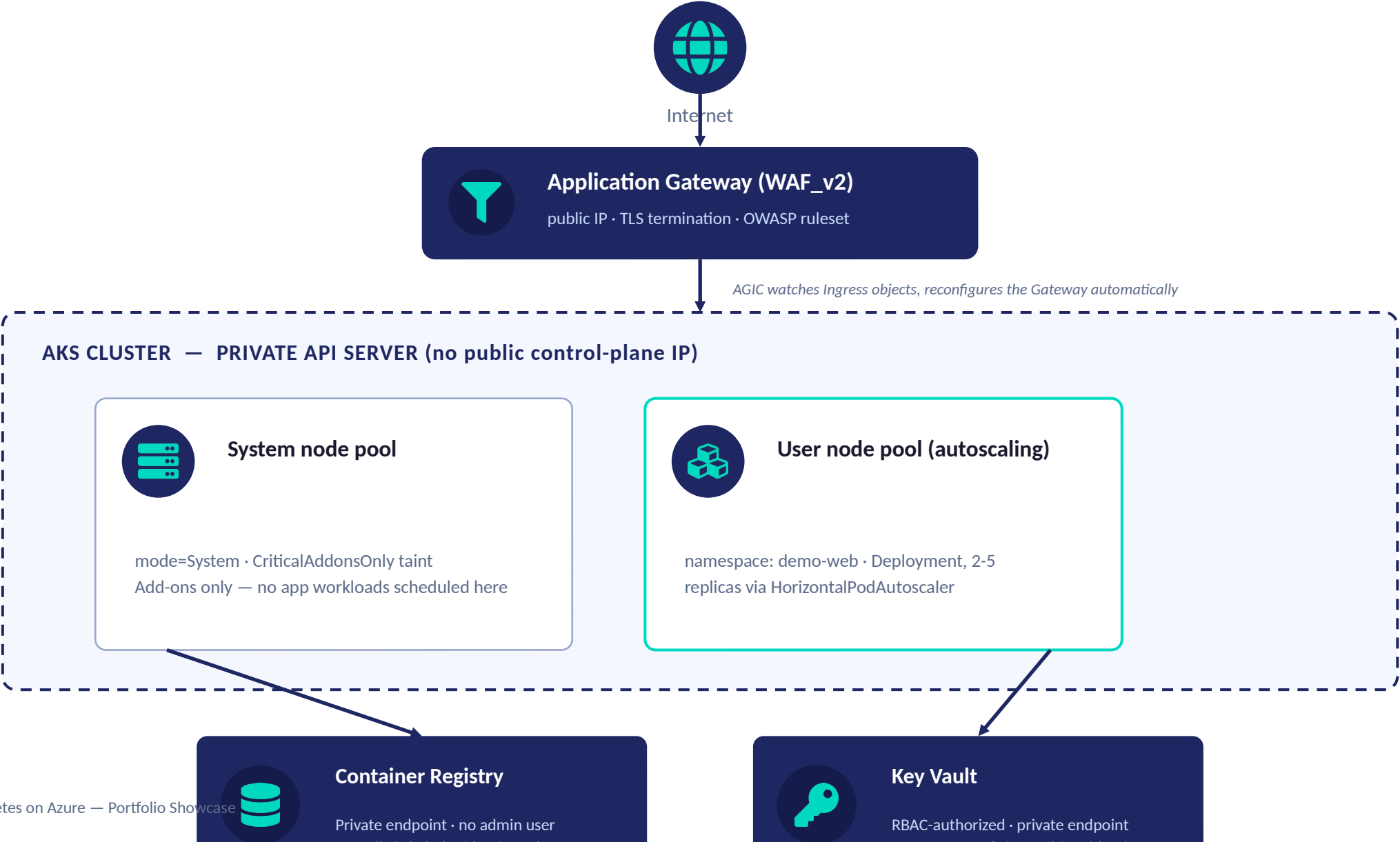


5-LAYER

Real incident,
diagnosed live

Both environments verified end-to-end as of 2026-09-18 — real pods, real TLS, real public IP, real HTTP 200.

Architecture



Inside the Cluster

Real Kubernetes objects, mapped to what they actually do here



Namespace

demo-web — isolates the app's objects, RBAC, and network policy



Deployment

2-5 replicas, non-root, read-only rootfs, all capabilities dropped



Service

Stable ClusterIP — what the Ingress and Gateway actually target



Ingress + AGIC

Declares routing intent; AGIC drives the real Application Gateway



HorizontalPodAutoscaler

Scales on CPU utilization (target 70%) with zero manual steps



NetworkPolicy (×3)

Default-deny; explicit allow only from the Gateway subnet + DNS



SecretProviderClass

Bridges the CSI driver to Key Vault — no cert material in Git

Security by Design



Private API server

No public control-plane IP, anywhere



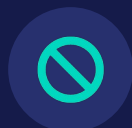
OIDC-only CI/CD

Federated tokens — zero stored Azure secrets in GitHub



Azure RBAC + Entra ID

Cluster access is group membership, not a static kubeconfig



Default-deny NetworkPolicy

Explicit allow only from the Gateway; egress limited to DNS



Non-root everywhere

Read-only rootfs, dropped capabilities, seccomp RuntimeDefault



No embedded credentials

ACR, Key Vault, and Gateway access all via managed identity

From git push to a running pod



Why this matters: a GitHub-hosted runner has no network line-of-sight into the cluster at all. “command invoke” executes kubectl inside the cluster's own control plane — zero VPN, self-hosted runner, or Bastion host required.



A Real Incident, Not a Demo

Prod's app sat in ContainerCreating for 24+ hours, undetected — diagnosed and fixed live, five layers deep.

1

Invalid vault name: "REPLACE_WITH_KEY_VAULT_NAME"

Prod's overlay was missing a patch dev already had

2

ForbiddenByRbac

Admin identity lacked Key Vault Certificates Officer on that vault

3

ForbiddenByConnection

Vault is private by design — no VPN/Bastion path in, even for an authorized identity

4

SecretNotFound (404)

The TLS certificate simply didn't exist in prod's vault yet

5

InvalidImageName

A manual kustomize apply bypassed CI's image-substitution step mid-recovery

Full root-cause timeline: docs/webplat-architecture.md → "Incident: prod SecretProviderClass never patched"

Kubernetes, Explained

Key terms, in plain language, in the context of this project

Pod

The smallest deployable unit — one or more containers sharing network/storage

Deployment

Declares “I want N copies running” and reconciles reality toward that

Service

A stable network identity for pods, even as pod IPs come and go

HPA

Horizontal Pod Autoscaler — changes replica count based on live metrics

Managed Identity

An Azure AD identity Azure manages the credential for — no key to leak

Node

A VM that runs pods — here, an Azure VM Scale Set instance

ReplicaSet

The Deployment's mechanism — a new one on every pod-template change

Ingress

A declarative routing rule; needs a controller (AGIC) to act on it

CSI driver

A plugin standard for mounting external storage/secrets into pods

Private cluster

The Kubernetes API server has no public IP at all

Where This Pattern Fits



Company websites with an SLA

Where “the site is down” is a business incident, not an inconvenience



Private line-of-business apps

Need network isolation but still a public-facing front door



Client engagement reference

Every piece answers a security-conscious procurement checklist



Multi-environment platforms

Each environment isolated at the network level, not just namespaced



Proof, Not Just Claims

Real command output, captured live from both environments

curl — prod

```
HTTP/1.1 200 OK
Server: nginx/1.27.5
```

```
$ curl .../healthz
ok
```

kubectl get pods — prod

```
demo-web-ccc969478-8cmfs
1/1 Running
demo-web-ccc969478-lmgm2
1/1 Running
```

App Gateway backend health

```
10.31.0.103 Healthy
10.31.0.136 Healthy

(both environments)
```

12 pieces of evidence in the repo

Resource groups · node pool health · private DNS zone · kubectl get nodes · pods running · backend health · end-to-end curl · cluster overview · Defender inventory — across both dev and prod, screenshots and raw command output alike.



Let's Talk

- ✓ Private AKS + Azure networking design
- ✓ Zero-secret CI/CD with OIDC federation
- ✓ Live production incident diagnosis
- ✓ Infrastructure as Code (Bicep) at scale

Sufyan Deen-Gabisi

github.com/sufideen/az-ent-sec-dashboard

Full technical write-up, evidence, and runbook in the repository's docs/ folder